Lab 10 — CSV và phân tích điểm sinh viên (lời giải từ PythoClass/main.py)

1. Đề bài tiếng Anh

LAB Session 10

Objective

Develop a Python program to manage student grades using **File I/O, Functions, and Error Handling**.

Requirements

Create a menu-driven program with these functions:

1. **Display All Students:** Read/display student data from `students.csv`.
2. **Add a New Student:** Enter student information and grades. Validate input and prevent duplicate Student IDs.
3. **Search Student by ID:** Search/display a student's information.
4. **Calculate Average Grade:** Display each student's average grade.
5. **Save Students to File:** Save current student data to `students.csv`.
6. **Exit:** Exit the program.

Error Handling

Handle file-not-found errors, invalid numeric input, grades outside **0–100**, and duplicate student IDs.

Sample `students.csv`

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Chuyển từ `Lab Session 10.docx` sang Markdown. Tên học viên trong dữ liệu ví dụ đã được thay bằng mã minh họa nhằm tránh công khai thông tin định danh; điểm số, lớp và cấu trúc bảng được giữ nguyên.

2. Đề bài tiếng Việt

LAB SESSION 10 — Quản lý điểm sinh viên bằng file CSV (tệp phân cách bằng dấu phẩy)

Mục tiêu

Xây dựng chương trình Python quản lý điểm sinh viên bằng thao tác **File I/O**, các **hàm** và **xử lý lỗi**.

Các yêu cầu

Tạo chương trình có menu với sáu chức năng:

1. **Hiển thị tất cả sinh viên:** Đọc dữ liệu sinh viên từ `students.csv` và hiển thị thông tin.
2. **Thêm sinh viên:** Nhập thông tin cá nhân và điểm của từng môn; kiểm tra dữ liệu hợp lệ và chống mã sinh viên trùng.
3. **Tìm theo mã sinh viên:** Nhập mã và hiển thị thông tin của sinh viên cần tìm.
4. **Tính điểm trung bình:** In điểm trung bình từng sinh viên.
5. **Lưu vào file:** Ghi dữ liệu đang quản lý vào `students.csv`.
6. **Thoát:** Kết thúc chương trình.

Xử lý lỗi

Cần xử lý **không tìm thấy file**, **nhập số không hợp lệ**, **điểm ngoài phạm vi 0–100** và **mã sinh viên trùng**.

Dữ liệu mẫu students.csv (tệp phân cách bằng dấu phẩy)

Dùng bảng CSV gồm mã sinh viên, tên, lớp, điểm Python, Math và ML. Bảng được giữ nguyên số liệu, tên đã được thay bằng mã/tên giả lập để bảo vệ danh tính:

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Các cột của file là `student_id,name,class_name,Python,Math,ML`; tên `Learner 01..10` là dữ liệu minh họa đã được ẩn danh.

3. Code Python có chú thích

```
import pandas as pd  # Pandas (thư viện xử lý bảng dữ liệu) với tên ngắn pd.

def inputHelper(data):  # Tạo hàm nhận mô tả dữ liệu nhập từ người dùng.
    label = data["label"]  # Lấy câu hướng dẫn nhập từ từ điển data.
    formats = data["formats"]  # Lấy danh sách kiểu dữ liệu được yêu câù.
    inputTemp = input(label)  # Hiển thị lời nhắc và đọc chuôĩ người dùng nhập.
    if inputTemp == "":  # Nêú người dùng đểtrống thì trả vềNone.
        return(None)  # Trả None: không có giá trị đâù vào.
    blocks =  inputTemp.split(";")  # Tách chuôĩ đâù vào tại dâú châḿ phẩy.

    # Check block
    if len(blocks) != len(formats):  # So sánh sôślượng phâǹ tửvới sôślượng kiểủ dự kiến.
        print("!!![ERROR] Input blocks is not correct!~")  # Hiển thị hoặc nhận thông tin trên terminal.

    # Check type
    for i in range(len(formats)):  # Duyệt từng vị trí cân kiểm tra.
        if type(blocks[i]).__name__ != formats[i]:  # LƯU Ý: split trả chuôĩ nên so kiểủ bằng tên không thực sự xác thực dữ liệu.
```

```
            print("!!![ERROR] Input format is not corrects")  # Hiển thị hoặc nhận thông tin trên terminal.
    return(blocks)  # Trả lại danh sách các chuôĩ đã được tách.

def displayAll():  # Chức năng 1: đọc và liệt kê tên sinh viên.
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    print("""
==== DISPLAY ALL STUDENT =====""")
    students = df[" name"].values  # Lấy cột ' name' có dấú cách đầu theo CSV gôć PythoClass.
    stt = 1  # Bắt đầu sô´thứ tự từ 1.
    for student in students:  # Duyệt từng tên sinh viên.
        print(f"{stt}. {student}")  # In sô´thứ tự và tên.
        stt += 1  # Tăng sô´thứ tự thêm một.

def studentSearch():  # Chức năng 2: tìm sinh viên bằng mã.
    print("""
==== SEARCH STUDENT =====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:  # Chỉ tiếp tục khi người dùng nhập dữ liệu.
        inputData = inputData[0]  # Lấý mã đầu tiên từ danh sách nhập.
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[df["student_id"] == inputData]  # Lọc các hàng có student_id trùng giá trị nhập.
        if len(result)!=0:  # Kiểm tra có dòng nào khớp hay không.
            print(result)  # In dòng dữ liệu đã tìm thấý.
        else:  # Nhánh xư lý trường hợp không thỏa điều kiện.
            print("!!![ERROR] Student not found.")  # Hiển thị hoặc nhận thông tin trên terminal.
        studentSearch()  # Gọi lại chính hàm tìm kiếḿ; lặp nhiều có thể tạo đệ quy quá sâu.
    else:  # Nhánh xư lý trường hợp không thỏa điều kiện.
        return  # Kết thúc hàm hiện tại.

def studentAverage():  # Chức năng 3: thống kê kết quả một sinh viên.
    print("""
==== STUDENT RESULT ====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:  # Chỉ tiếp tục khi người dùng nhập dữ liệu.
        inputData = inputData[0]  # Lấý mã đầu tiên từ danh sách nhập.
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[(df["student_id"] == inputData)]  # Lọc đúng sinh viên bằng mã.
        if len(result)!=0:  # Kiểm tra có dòng nào khớp hay không.
            values = result.values  # Đổi bảng kết quả sang mảng NumPy để lấý theo chỉ sô´
            print(f"""
Student name: {values[0][1]}
Python score: {values[0][3]}
Math score: {values[0][4]}
ML score: {values[0][5]}
Average score:{round((values[0][3] + values[0][4] + values[0][5])/3,2)}
Highest score:{max(values[0][3],values[0][4],values[0][5])}
Lowest score:{min(values[0][3],values[0][4],values[0][5])}
""")
        else:  # Nhánh xư lý trường hợp không thỏa điều kiện.
            print("!!![ERROR] Student not found.")  # Hiển thị hoặc nhận thông tin trên terminal.
        studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
    else:  # Nhánh xư lý trường hợp không thỏa điều kiện.
```

```
        return  # Kết thúc hàm hiện tại.

def analyzeSubject():  # Chức năng 4: thống kê tổng/trung bình/cao nhất/thấp nhất từng môn.
    print("""
==== SUBJECT PERFORMANCE ====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    data = {
        "Average":[df["Python"].mean(),df["Math"].mean(),df["ML"].mean()],
        "Total":[df["Python"].sum(),df["Math"].sum(),df["ML"].sum()],
        "Highest":[df["Python"].max(),df["Math"].max(),df["ML"].max()],
        "Lowest":[df["Python"].min(),df["Math"].min(),df["ML"].min()],
    }
    dfShow = pd.DataFrame(data,index=["Python","Math","ML"])  # Tạo bảng với ba hàng tương ứng Python, Math và ML.
    print(dfShow)  # In bảng tổng hợp theo môn.


def analyzeClass():  # Chức năng 5: phân tích sô´sinh viên và trung bình từng lớp.
    print("""
===== CLASS PERFORMANCE =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    studentCount =  df.groupby("class_name")["student_id"].count()  # Nhóm theo lớp và đếm mã sinh viên.
    pythonMean =  df.groupby("class_name")["Python"].mean()  # Tính điểm Python trung bình theo lớp.
    mathMean =  df.groupby("class_name")["Math"].mean()  # Tính điểm Math trung bình theo lớp.
    mlMean =  df.groupby("class_name")["ML"].mean()  # Tính điểm ML trung bình theo lớp.
    result = pd.merge(studentCount,pythonMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mathMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mlMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result["Overall Average"] = (result["Python"] + result["Math"] +  result["ML"])/3  # Trung bình chung từ trung bình ba môn.


    result.reset_index(inplace=True) #Trợ giúp của chat GPT ạ!~
    result.rename(columns={"student_id":"Students"}, inplace=True) #Trợ giúp của chat GPT ạ!~
    result.rename(columns={"class_name":"Class"}, inplace=True)  # Đổi tiêu đề cột để bảng dễ đọc.

    result = result.round(2) # Trợ giúp của chat GPT ạ!~

    print(result)  # In dòng dữ liệu đã tìm thấy.

def top3():  # Chức năng 6: xếp hạng ba sinh viên có điểm trung bình cao nhất.
    print("""
===== TOP 3 STUDENTS =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    df["Average"] =   (df["Python"] + df["Math"] +  df["ML"])/3  # Thêm cột Average bằng trung bình Python, Math và ML.
    dfSorted = df.sort_values(by="Average",ascending=False)  # Sắp xếp theo Average giảm dần.
    dfSorted.drop(columns=["Python","Math","ML"], inplace=True) # Trợ giúp của chat GPT ạ!~
    dfSorted.rename(columns={"class_name":"Class","student_id":"Student ID"}, inplace=True)  # Đổi tên cột cho dễ hiểu.
    dfSorted = dfSorted.round(2)  # Làm tròn điểm trung bình.

    print(dfSorted.head(3))  # Chỉ hiển thị ba hàng đứng đầu.

# MENU()
def menu():  # Xây dựng menu tương tác lặp đến khi chọn thoát.
    choice = 0  # Khởi tạo lựa chọn mặc định.
    while True:  # Lặp menu cho tới khi gặp break.
        print("""
================================
 STUDENT PERFORMANCE ANALYSIS
--------------------------------------------------------
```

```
1. Display all students
2. Search for a student
3. Calculate student average
4. Analyze subject performance
5. Analyze class performance
6. Show top 3 students
7. Exit
        """)
        # nhap lua chon
        inputData =input("Please select (1-7): ")  # Nhận lựa chọn của người dùng từ 1 đến 7.
        if inputData.isnumeric():  # Kiểm tra chuỗi chỉ có chữ số trước khi ép kiểu.
            choice = int(inputData)  # Chuyển lựa chọn từ chuỗi sang số nguyên.
            if choice == 1:  # Điều hướng tới chức năng đã chọn.
                displayAll()
            elif choice == 2:  # Xử lý lựa chọn còn lại.
                studentSearch()  # Gọi lại chính hàm tìm kiếm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 3:  # Xử lý lựa chọn còn lại.
                studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 4:  # Xử lý lựa chọn còn lại.
                analyzeSubject()
            elif choice == 5:  # Xử lý lựa chọn còn lại.
                analyzeClass()
            elif choice == 6:  # Xử lý lựa chọn còn lại.
                top3()
            elif choice == 7:  # Xử lý lựa chọn còn lại.
                break  # Dừng vòng lặp menu.
            else:  # Nhánh xử lý trường hợp không thỏa điều kiện.
                print("!!![ERROR] Please enter from 1 to 7")  # Hiển thị hoặc nhận thông tin trên terminal.
        else:  # Nhánh xử lý trường hợp không thỏa điều kiện.
            print("!!![ERROR] Please enter from 1 to 7")  # Hiển thị hoặc nhận thông tin trên terminal.

# goi ham
menu()  # Khởi chạy menu chính khi chạy file.
```

4. Giải thích nội dung và cách giải

Tổng quan chương trình thực tế của `main.py`

Tệp này là **bài giải đang có trong repo PythoClass**, sử dụng `pandas` để đọc và phân tích `students.csv`. Chương trình in menu **7 lựa chọn**: hiển thị, tìm sinh viên, thống kê một sinh viên, thống kê theo môn, theo lớp, xếp hạng 3 sinh viên đầu và thoát. Trong khi đó, đề Word Lab 10 yêu cầu menu **6 lựa chọn**, có **thêm sinh viên** và **lưu file**. Đây là **hai phạm vi không hoàn toàn giống nhau**, nên không được coi `main.py` đã đáp ứng toàn bộ đề gốc.

Đọc dữ liệu và cách chương trình chạy

1. **`import pandas as pd`** nạp thư viện Pandas. `pd.read_csv("students.csv")` đọc một file bảng từ **thư mục chạy hiện tại**. Nếu file không có ở đó, lệnh đọc phát sinh `FileNotFoundError`.
2. **`inputHelper(data)`** hiển thị lời nhắc rồi dùng `split(";")` để tách nhiều giá trị. Một mục nhập trống trả về `None`; các hàm tìm kiếm và tính điểm dùng điều này để kết thúc.
3. **`displayAll()`** đọc bảng, lấy cột tên rồi in từng học viên bằng `for` và số thứ tự.
4. **`studentSearch()`** lọc bảng bằng biểu thức Boolean `df["student_id"] == inputData`; nếu lọc ra hàng thì in, nếu không thì báo không tìm thấy.
5. **`studentAverage()`** lấy điểm Python, Math, ML của một học viên, tính **trung bình = tổng ba điểm chia 3**, dùng `round(..., 2)` để hiển thị hai chữ số, `max(...)` / `min(...)` để lấy điểm cao/thấp nhất.
6. **`analyzeSubject()`** gọi `.mean()`, `.sum()`, `.max()`, `.min()` trên từng cột điểm; `pd.DataFrame(data, index=...)` trình bày các thống kê theo môn.

7. **`analyzeClass()`** gọi `groupby("class_name")` để chia theo lớp, `.count()` để đếm sinh viên, `.mean()` tính trung bình; `pd.merge(..., on="class_name", how="outer")` ghép bảng mà vẫn giữ các nhóm lớp, `reset_index(inplace=True)` chuyển chỉ số thành cột, `rename(columns=..., inplace=True)` đổi tên cột, `round(2)` làm tròn.
8. **`top3()`** thêm cột `Average`, gọi `sort_values(by="Average", ascending=False)` để xếp cao xuống thấp; `drop(columns=..., inplace=True)` bớt cột thừa và `head(3)` lấy ba hàng đầu.
9. **`menu()`** dùng `while True`, `if/elif` và `break` để điều hướng 7 lựa chọn.

Những điểm phải biết khi tự chạy và đối chiếu đề

Tên cột `" name"` có dấu cách: CSV gốc trong PythoClass có header `student_id, name,class_name,Python,Math,ML`, nghĩa là tên cột thực tế là `" name"` có khoảng trắng đầu. Bởi vậy `df[" name"]` trong `displayAll()` chạy với **CSV gốc**, nhưng sẽ lỗi `KeyError` nếu dùng dữ liệu mẫu trong đề Word đã chuẩn hóa thành `"name"` không có dấu cách. Khi thay CSV, cần nhất quán tên cột.

`inputHelper()` có hạn chế: `input().split(";")` luôn trả các chuỗi `str`. Cách kiểm tra `type(blocks[i]).__name__` không ép sang số thực/nguyên, và nhánh báo sai số trường không dừng trước vòng lặp: nếu thiếu trường có thể xảy ra `IndexError`. Nên kiểm tra số trường rồi `return` hoặc báo lỗi, sau đó chuyển kiểu thực sự bằng `int()/float()` có xử lý `ValueError` khi mở rộng chức năng.

Gọi đệ quy để hỏi lại: `studentSearch()` và `studentAverage()` gọi lại chính nó sau mỗi lượt tra cứu. Cách này có thể dùng nhanh nhưng gọi nhiều lần có nguy cơ `RecursionError`; vòng `while` là lựa chọn an toàn hơn.

Đối chiếu với đề Word: `main.py` **không có** thao tác thêm sinh viên, kiểm tra mã trùng khi thêm, lưu sinh viên ra CSV và kiểm tra điểm 0–100. Do đó, nếu bài thi yêu cầu đủ các chức năng của Word, vẫn phải bổ sung chúng. Phiên bản `labs/pythoclass/LAB-10-file-csv/SOLUTION.py` là **lời giải tham khảo khác** đã triển khai các yêu cầu đó; không nên nhầm nó với `main.py` của bạn.

Bảo toàn nguồn: Phần code in phía trên giữ nguyên **các dòng lệnh thực thi** trong `PythoClass/main.py`; chỉ bổ sung chú thích tiếng Việt cuối dòng để ôn tập. Những vấn đề được phân tích ở đây thay vì âm thầm sửa logic chương trình. Dùng `students.csv` gốc cùng thư mục chạy khi muốn kiểm tra đúng hành vi mã nguồn.